



# 深度学习与计算机视觉

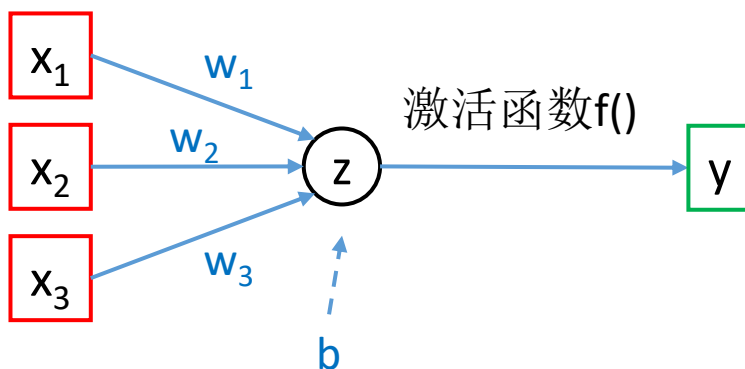
---

## 搭建全连接层网络， 实现图像分类

王一帆



## ■ 单个神经元的数学表示过程

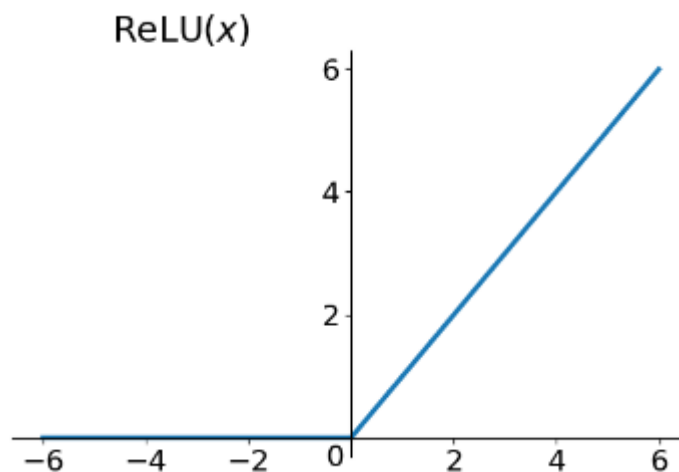


$$z = (w_1x_1 + w_2x_2 + w_3x_3) + b$$
$$y = f(z)$$

## ■ 激活函数

- ReLU

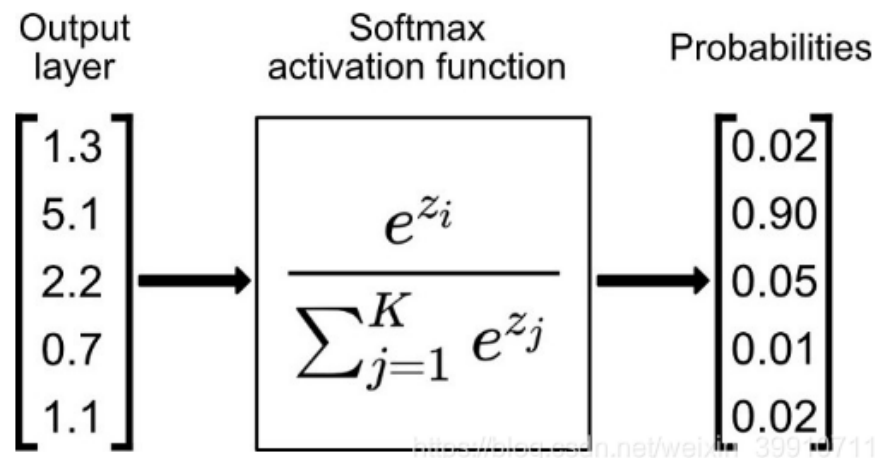
$$\sigma(x) = \begin{cases} \max(0, x) & , x \geq 0 \\ 0 & , x < 0 \end{cases}$$



## ■ 激活函数

- Softmax

Softmax 是用于多类分类问题的激活函数，在多类分类问题中，超过两个类标签则需要类成员关系。对于长度为  $K$  的任意实向量，Softmax 可以将其压缩为长度为  $K$ ，值在  $(0, 1)$  范围内，并且向量中元素的总和为 1 的实向量（**可以理解为概率**）。





## ■ 激活函数

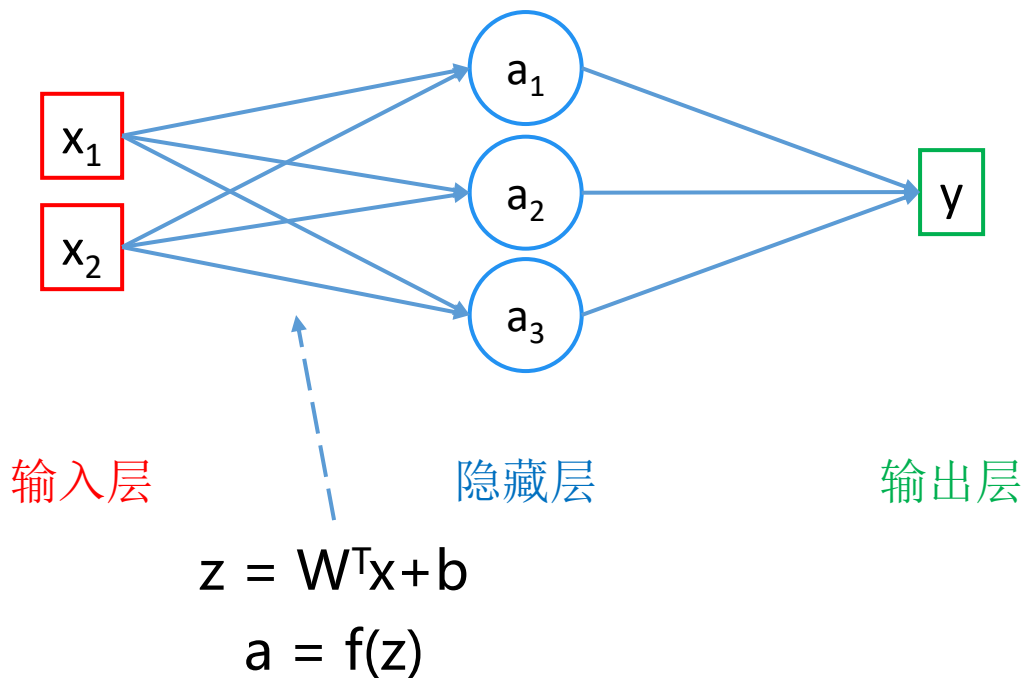
- nn.Softmax(dim)

dim: 可选参数，表示要进行 softmax 的维度，默认值为 None，表示对最后一维进行 softmax。

```
softmax = nn.Softmax(dim=1)
score = softmax(score)    # 利用 softmax 将 网络输出结果转化为是分类的概率向量
```

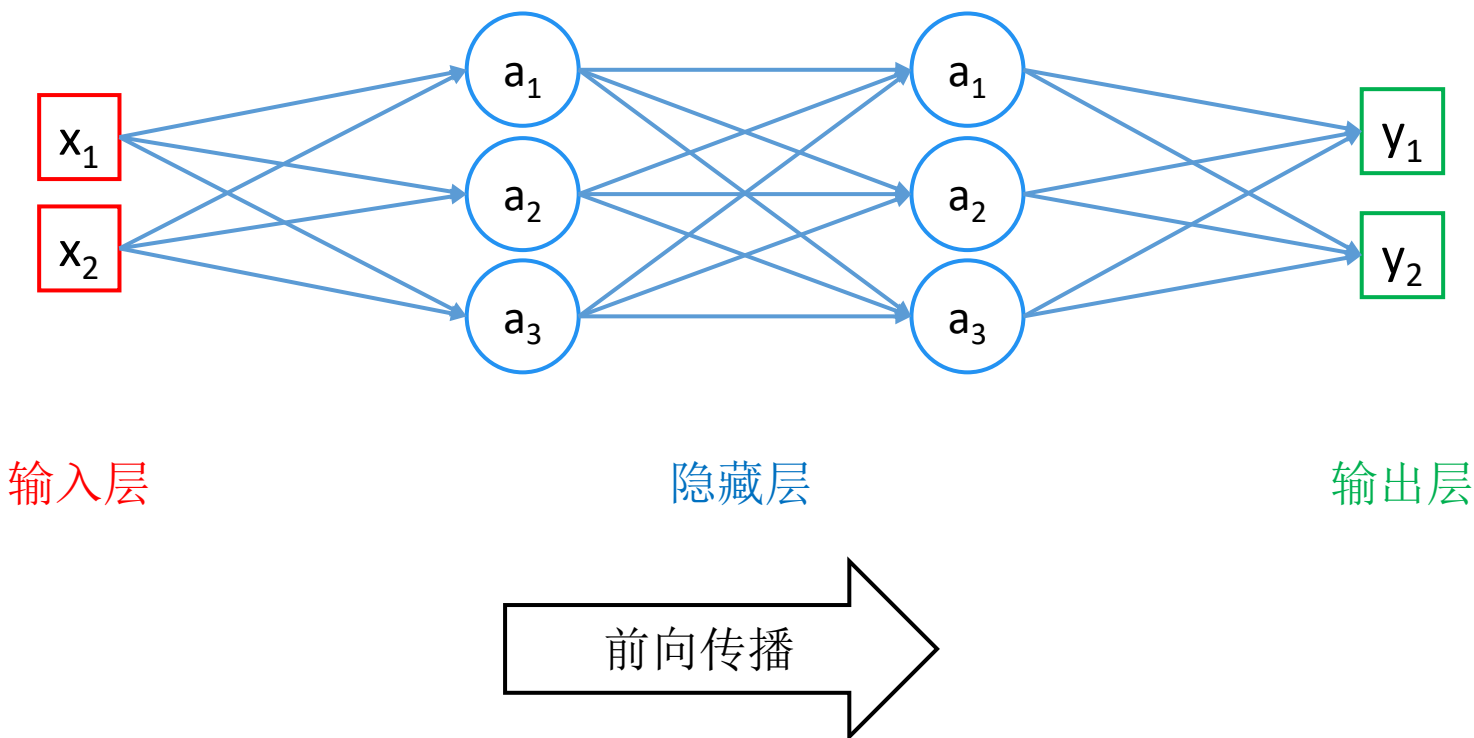
## ■ 全连接神经网络

隐藏层有多个节点，每一层的每一个节点都与上下层节点全部连接，这种神经网络称作全连接网络。



## ■ 全连接神经网络

多层全连接神经网络



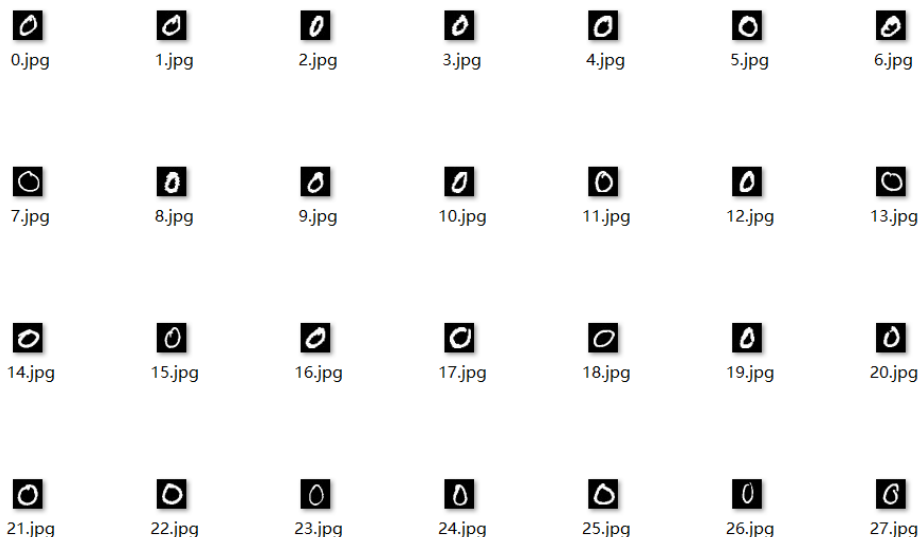
## ■ 实战：搭建网络实现图像分类

搭建一个简单的全连接神经网络实现图像分类可分为以下步骤：

1. 准备数据
2. 搭建网络
3. 载入权重
4. 图片预处理
5. 推理预测

## ■ 准备数据

对于深度学习的任何项目来说，数据集是其中最为关键的部分。**MNIST**数据集是美国国家标准与技术研究院收集整理的大型手写数字数据库,包含60,000个示例的训练集以及10,000个示例的测试集。其中的图像的尺寸为28\*28，具体内容为手写数字0-9。



## ■ 搭建网络

- 问题分析：

多分类问题，输入为 $28 \times 28$ 的单通道灰度图像，输出10个类别其中之一

- 网络设计：

输入层：将 $28 \times 28$ 展平成 $1 \times 784$ 向量输入

隐藏层：多层不同节点数量的全连接神经网络

输出层：10个结点，即输出十个类别的预测概率

## ■ 搭建网络

创建自定义的数据集需要继承torch下的nn.Module类  
实现两个方法：

- `__init__`：初始化全连接神经网络
- `forward`：前向传播

```
class MNIST_Net(nn.Module):  
  
    def __init__(self):  
        super().__init__()  
        # 网络结构  
  
    def forward(self, x):  
        # 前向传播过程
```

## ■ 搭建网络

- nn.Sequential(网络模块)

将多层网络模块封装成一个模块，使用前从torch中import，如：

```
1  from torch import nn
2
3  class MNIST_Net(nn.Module):
4
5      def __init__(self):
6          super().__init__()
7
8          self.net = nn.Sequential(
9              nn.Linear(784, 512),
10             nn.ReLU(),
11             nn.Linear(512, 10),
12             nn.ReLU(),
13             nn.Softmax(dim=1)
14         )
```

## ■ 搭建网络

- `nn.Linear(in_features, out_features, *bias)`

`in_features`: 输入神经元个数

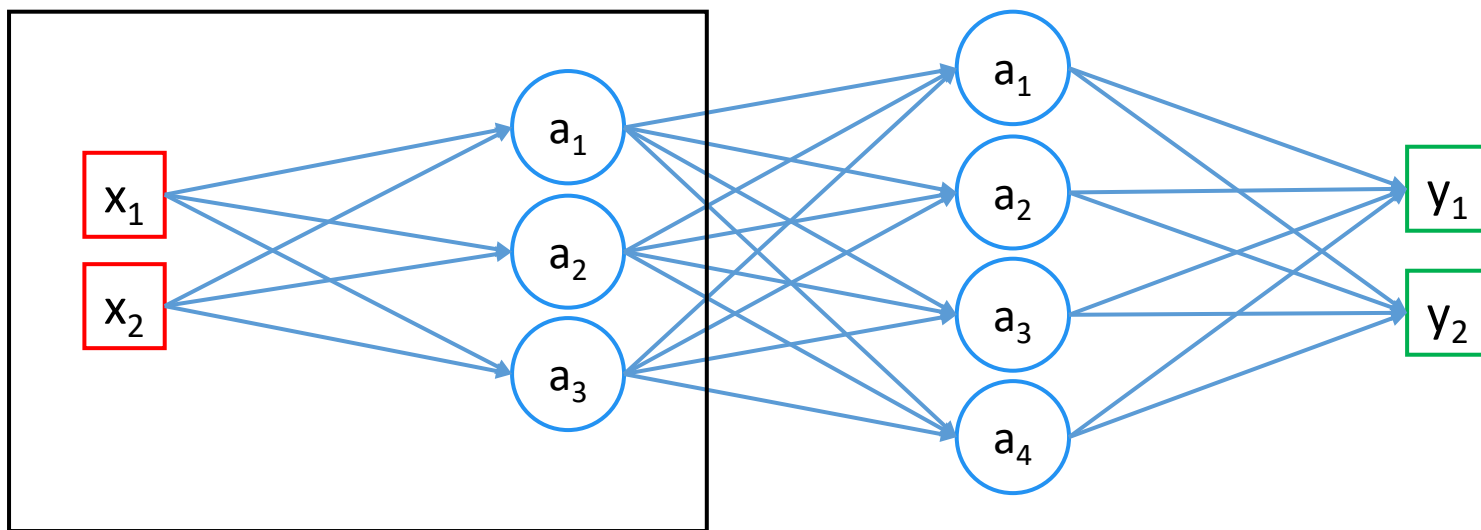
`out_features`: 输出神经元个数

`bias`: 可选参数，是否添加偏置，默认为True（建议选默认值即可）



## ■ 搭建网络

- `nn.Linear(in_features, out_features)`

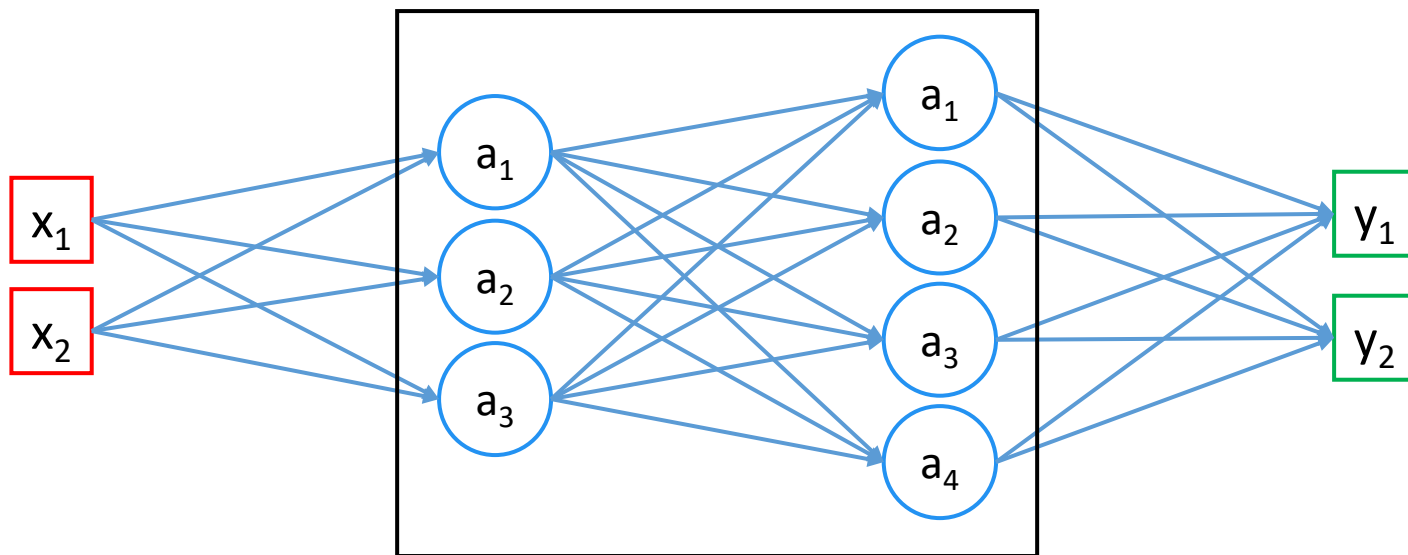


`in_features=2, out_features=3`



## ■ 搭建网络

- `nn.Linear(in_features, out_features)`

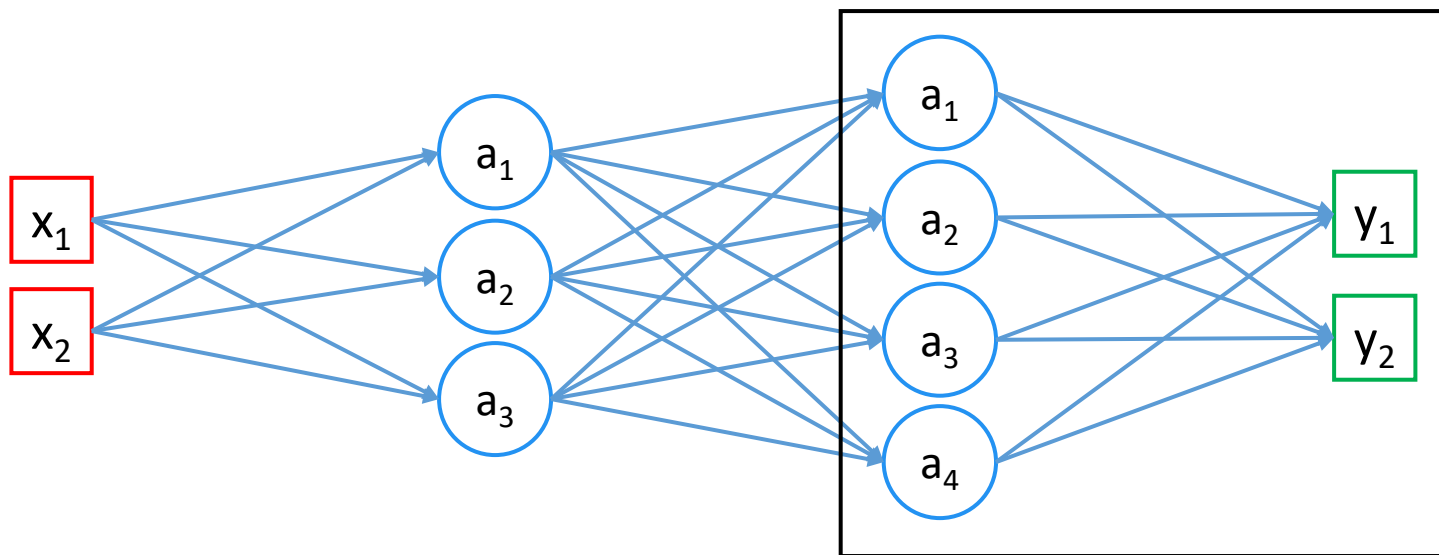


`in_features=3, out_features=4`



## ■ 搭建网络

- `nn.Linear(in_features, out_features)`



`in_features=4, out_features=2`



## ■ 搭建网络

- `nn.ReLU(*inplace)`

我们选择ReLU函数作为激活函数

`inplace`: 可选参数，是否改变输入数据，如果设置为True，则会直接修改输入数据；如果设置为False，则不对输入数据做修改。默认为False。

## ■ 搭建网络

我们的网络隐藏层为4层：

|        | 输入元个数 | 输出元个数 |
|--------|-------|-------|
| layer1 | 784   | 512   |
| layer2 | 512   | 256   |
| layer3 | 256   | 64    |
| layer4 | 64    | 10    |



## ■ 搭建网络

每层的结构为nn.Linear和nn.ReLU，我们将其封装到nn.Sequential中，如layer1为：

```
self.layer1 = nn.Sequential(  
    nn.Linear(784, 512),  
    nn.ReLU()  
)
```

## ■ 搭建网络

- 前向传播 (forward)

定义前向传播过程，完成网络搭建

```
# 进行前向传播返回结果
def forward(self, x):
    x = self.layer1(x)
    x = self.layer2(x)
    x = self.layer3(x)
    x = self.layer4(x)
    x = self.softmax(x)
    return x
```



## ■ 载入权重

- `torch.load(f, map_location)`

f: 权重文件路径

map\_location: 可选参数, 表示存储权重的device (CPU或GPU)

- `model.load_state_dict(权重)`

将`torch.load()`加载的权重载入到model



## ■ 载入权重

```
68 ▶ if __name__ == '__main__':  
69     # 初始化网络  
70     net = MNIST_Net()  
71     # 载入权重  
72     net.load_state_dict(torch.load('ckpt.pt', map_location='cpu'))
```

## ■ 图片预处理

输入网络的图像需要和网络训练时的预处理格式一致，如：

- 读入灰度图像（**单通道**）
- 将 $28 \times 28$ 图像展平成 $1 \times 784$
- 归一化，将数值范围控制在0-1之间
- 转化成torch，保证网络可以接收输入数据进行推理计算

```
# 数据预处理
path = 'MNIST_IMG/TEST/0/0.jpg'
test_img = cv2.imread(path, 0) # 读入灰度图片
test_img = test_img.reshape(1, 784) # 展平
test_img = np.float32(test_img / 255) # 归一化
test_img = torch.from_numpy(test_img) # 转化成torch格式
```

## ■ 推理预测

- `score = net(test_img)`

输入数据经过net中定义的forward过程，输出十个类别的预测概率

```
tensor([[1.0000e+00, 4.6354e-26, 4.6354e-26, 4.6354e-26, 4.6354e-26, 4.6354e-26,  
         4.6354e-26, 4.6354e-26, 4.6354e-26, 4.6354e-26]],  
        grad_fn=<SoftmaxBackward0>)
```



## ■ 推理预测

- `torch.argmax(input, *dim)`

输入一个tensor, 返回最大值索引

input: 输入的tensor

dim: 选择从哪个维度上计算最大值

我们用`answer = torch.argmax(score, dim=1)`对`1*10`的score (十个数字的预测概率得分) 取最大值返回索引, 作为最终答案

```
answer = torch.argmax(score, dim=1)
print(answer)
```

```
tensor([1])
```

进程已结束, 退出代码0